

CAHIER DE CHARGE

1. Présentation Générale du Projet

Nom du projet : API de Suivi et de Valorisation des Produits Agricoles Locaux

Objectif principal : Offrir une plateforme centralisée permettant de connecter les producteurs agricoles gabonais aux marchés et aux consommateurs, tout en assurant la traçabilité, la transparence et la visibilité des produits locaux.

Public cible :

- Agriculteurs individuels et coopératives
 - Commerçants et restaurateurs
 - Marchés et points de vente locaux
 - Consommateurs
 - Autorités et organismes publics (ministères de l'agriculture.)
-

2. Fonctionnalités Clés de l'API

2.1 Authentification & Gestion des Utilisateurs

- **Inscription / Connexion (JWT)**
- **Rôles utilisateurs :**
 - Producteur
 - Administrateur
- **Sécurité des mots de passe :** bcryptjs
- **Gestion du profil :** nom, adresse, numéro de téléphone, localisation, type d'exploitation (individuelle / coopérative), certifications

2.2 Gestion des Exploitations Agricoles

- Création et mise à jour d'une fiche d'exploitation : superficie, type de cultures, description
- Liée au profil utilisateur

2.3 Gestion des Produits

- CRUD des produits référencés (manioc, banane plantain, igname, etc.)
- Données : nom, description, image, saisonnalité, méthode de culture
- Association des produits aux producteurs

2.4 Gestion des Lots de Production

- Création de lots : produit, poids, date de récolte, qualité, statut (disponible / vendu)
- Génération d'un identifiant unique par lot
- Traçabilité depuis la récolte jusqu'à la vente

2.5 Suivi des Stocks

- Mise à jour des quantités disponibles
- Indication du statut de stock : en stock / rupture / en attente
- Définition des périodes de disponibilité

2.6 Référencement des Points de Vente

- Création de points de vente : nom, description, géolocalisation, horaires
- Lier les produits ou les producteurs à ces points
- Recherche géographique de points de vente proches

3. Architecture Technique

3.1 Stack Technologique

- **Backend** : Node.js + Express.js
- **Base de données** : MySQL
- **ORM** : Prisma
- **Authentification** : JSON Web Tokens (JWT)
- **Sécurité des mots de passe** : bcryptjs
- **Conteneurisation** : Docker
- **Documentation** : Swagger
- **Déploiement prévu** : Render

3.2 Architecture RESTful

Méthode	Endpoint	Description
POST	<code>/utilisateur</code>	Inscription utilisateur
POST	<code>/utilisateur/login</code>	Connexion utilisateur
GET	<code>/utilisateur</code>	Récupérer tous les utilisateur
GET	<code>/utilisateur/:id</code>	Récupérer profil utilisateur
PUT	<code>/utilisateur/:id</code>	Modifier profil utilisateur
POST	<code>/exploitations/</code>	Créer une exploitation agricole
GET	<code>/exploitations/:id</code>	Voir détails d'une exploitation
POST	<code>/produits/</code>	Ajouter un produit au catalogue
GET	<code>/produits/</code>	Lister les produits disponibles

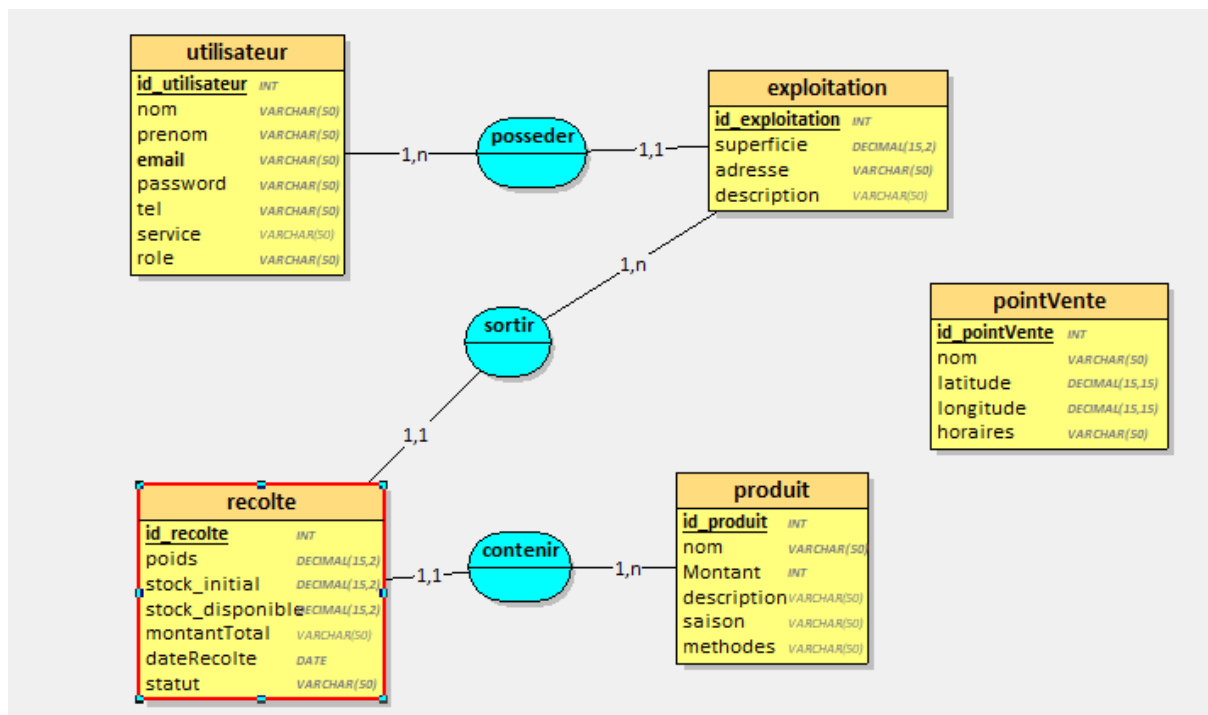
POST /*recolte*/ Créer un récolte de production

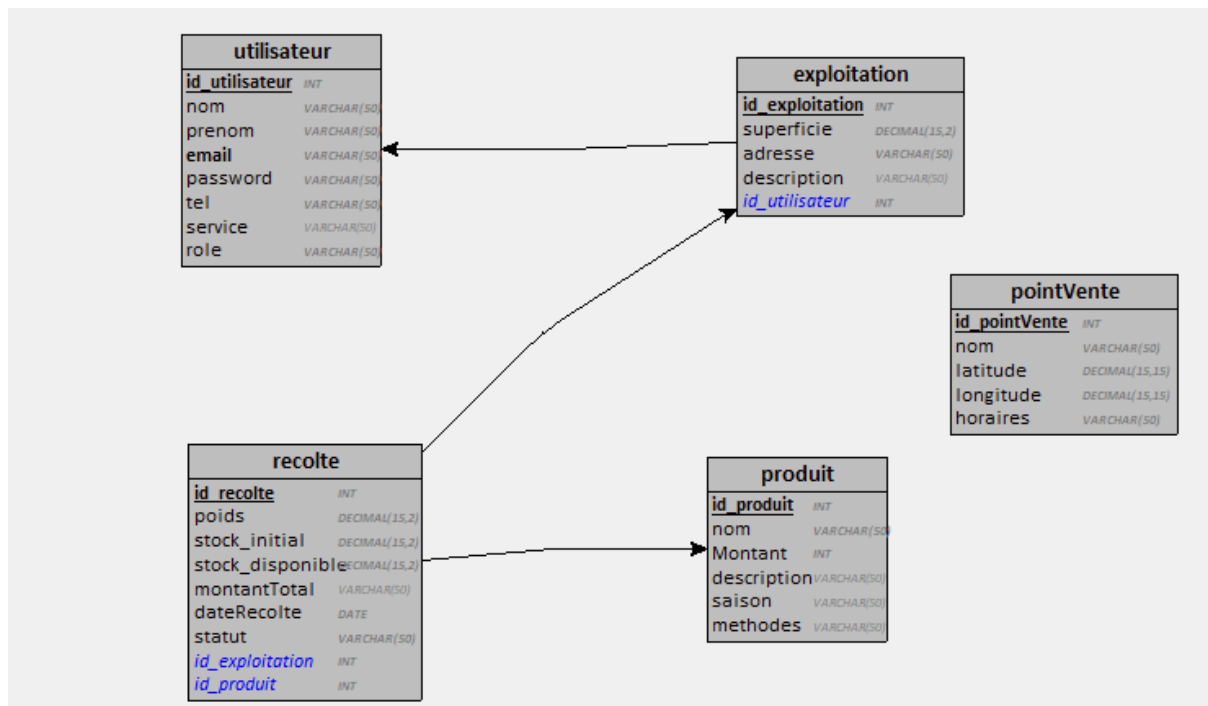
GET /*recolte* Récupérer la liste des récolte

POST /*pointVente*/ Enregistrer un point de vente

GET /*pointVente/proche* Trouver les points proches d'une position

4. Modélisation de la Base de Données





5. Contraintes Techniques et Sécurité

- Respect des bonnes pratiques REST
 - Séparation claire des rôles via middleware
 - Protection contre les injections SQL via Prisma
 - Hachage des mots de passe avec bcryptjs
 - Sécurisation des routes avec JWT
-

6. Livrables Attendues

1. Code source de l'API avec toute la logique métier
2. Fichier `schema.prisma` et script SQL d'initialisation
3. Dockerfile et fichier `.env`

4. Documentation Swagger interactive
 5. Déploiement (ex : sur Render)
 6. Cahier des charges (ce document)
-

7. Évolutions Futures Possibles

- Application mobile ou frontend Next.js
- Système de messagerie entre producteurs et acheteurs
- Intégration d'un système de paiement local (Airtel Money, Moov Money)
- QR code sur les lots pour la traçabilité en boutique